

Jour 3 : CNN, classification d'images, transfer learning

Aujourd'hui, notre objectif va être de toucher du 95% d'accuracy sur un dataset de 25 000 images de chats et de chiens. En partant de zéro.

Au programme ce matin, on va comprendre comment fonctionnent les réseaux convolutifs : on calcule nous-mêmes les shapes couche par couche, et on voit pourquoi un [CNN](#) "regarde" une image d'une façon qu'un PMC ne pourra jamais reproduire.

L'après-midi, on code trois itérations du même problème. La première plafonne à 70% et c'est voulu. La deuxième monte à 85% avec de l'augmentation et du dropout. La troisième, avec MobileNetV2 et du transfer learning, atteint 95% en quelques epochs. Le contraste, c'est ça le fil rouge de la journée.

1 - Révisions rapides

Reprenons les points clés de J2 avant d'attaquer les images.

Hier, sur le dataset Pima Diabetes, est-ce que quelqu'un a observé de l'overfitting dans ses courbes TensorBoard ? À quoi ça ressemblait ?

"la val_loss qui remonte", "l'écart entre train et val qui se creuse", "le modèle qui mémorise au lieu de généraliser".

Ce qu'on retient de J2 :

- **TensorBoard** : lancer depuis Colab avec `%load_ext tensorboard + %tensorboard --logdir logs/`.
Les courbes train/val en temps réel, c'est le tableau de bord de tout ML engineer.
- **Overfitting vs underfitting** :
Overfitting = les courbes train et val divergent (train descend, val remonte ou stagne).
Underfitting = les deux courbes restent hautes, le modèle n'a pas assez appris.
- **Régularisation** : L1 (pénalise les poids non nuls), L2 (pénalise les grands poids), Dropout (éteint des neurones aléatoirement pendant l'entraînement), Early Stopping (on coupe quand la val_loss ne s'améliore plus).
- **keras-tuner** : recherche automatique des hyperparamètres. Demain on reparle des séquences, mais le principe (espace de recherche défini, oracle qui choisit) reste le même.

Aujourd'hui, on applique tout ça sur un nouveau type de données : les images. Et les problèmes d'overfitting vont être encore plus visibles.

Planning de la Journée

1. Révisions rapides
2. Pourquoi les convolutions ?

3. Architecture CNN : shapes étape par étape
4. Surmonter l'overfitting image : augmentation + dropout
5. Transfer learning et architectures de référence
6. Comparaison Keras / PyTorch
7. **Après-midi : TP Cats vs Dogs** (phases 1.1 à 3.5)

2 - Pourquoi les convolutions ?

L'essentiel

Imaginez une image 224x224 pixels en couleur (RGB). Si on l'aplatit et qu'on la connecte à 512 neurones comme en J1 avec MNIST, combien de poids est-ce que ça fait pour la première couche seule ?

$224 \times 224 \times 3 = 150\,528$ entrées. $\times 512$ neurones = **77 millions de poids**. Pour une seule couche. Sur une seule taille d'image. Et si vous avez 4 couches cachées, vous êtes à des centaines de millions de paramètres pour un réseau qui ne marche pas encore.

Mais le vrai problème n'est pas la taille. C'est **l'invariance**.

Imaginez un chat en haut à gauche de l'image. Votre PMC apprend que "quand les pixels 12, 13, 14 sont activés d'une certaine façon, c'est un chat". Si le même chat est en bas à droite, les pixels 12, 13, 14 sont éteints. Le PMC n'a rien appris en commun. Il doit tout réapprendre pour chaque position possible. C'est le problème de l'invariance de translation : un PMC en est incapable, sauf à mémoriser chaque position.

La **convolution** résout les deux problèmes en même temps.

Localité : au lieu de connecter chaque neurone à toute l'image, on le connecte à un petit patch local, par exemple 3x3 pixels. Il ne "voit" que son voisinage immédiat.

Partage de poids : le même filtre (kernel) est appliqué à toute l'image. Si le filtre détecte des bords horizontaux, il les détecte partout, pas juste en haut à gauche. Un seul jeu de poids, utilisé à chaque position.

Invariance de translation : le chat à gauche et le chat à droite activent le même filtre dans des endroits différents de la feature map. Le réseau n'a pas besoin de réapprendre.

La convolution en pratique

Un **kernel** (ou filtre) est une petite matrice, généralement 3x3 ou 5x5, qui "glisse" sur l'image. À chaque position, on calcule le produit élément par élément entre le kernel et le patch de l'image, puis on somme. Le résultat est un seul nombre qui dit "à quel point ce motif est présent ici".

On va dérouler ce calcul entièrement à la main, une fois, sur un exemple minuscule. Pas pour le refaire à chaque image (Keras s'en charge), mais pour qu'après ça la convolution ne soit plus une boîte noire.

La seule chose à retenir en sortie : un kernel qui glisse sur une image produit une nouvelle image, la feature map, avec une valeur par position.

```
KERNEL 3x3 GLISSANT SUR IMAGE 7x7 (padding=valid, stride=1)
```

```
Résultat : feature map 5x5 (formule : (7 - 3 + 0) / 1 + 1 = 5)
```

IMAGE 7x7 (valeurs illustratives)

```
1 0 2 3 1 0 1
0 1 1 2 0 1 0
2 1 3 1 2 1 0
1 0 1 0 1 0 2
0 2 1 2 0 1 1
1 1 0 1 2 0 0
0 1 2 0 1 1 1
```

KERNEL 3x3 (poids appris)

```
w1 w2 w3      1 -1 0
w4 w5 w6  =   -1 2 1
w7 w8 w9      0 1 -1
```

POSITION 1 (i=0, j=0) → feature_map[0, 0]

```
┌──────────┐
│ 1 0 2 | 3 1 0 1
│ 0 1 1 | 2 0 1 0
│ 2 1 3 | 1 2 1 0
└──────────┘
```

Patch 3x3 extrait : $[[1,0,2],[0,1,1],[2,1,3]]$

```
1 0 1 0 1 0 2
0 2 1 2 0 1 1
(3x-1)
1 1 0 1 2 0 0
0 1 2 0 1 1 1
```

Calcul : $(1 \times 1) + (0 \times -1) + (2 \times 0) + (0 \times -1) + (1 \times 2) + (1 \times 1) + (2 \times 0) + (1 \times 1) +$

$= 1 + 0 + 0 + 0 + 2 + 1 + 0 + 1 - 3 = 2$

feature_map[0, 0] = 2 (avant ReLU)

POSITION 2 (i=0, j=1) → feature_map[0, 1]

```
1 ┌──────────┐
0 │ 0 2 3 | 1 0 1
2 │ 1 1 2 | 0 1 0
1 │ 1 3 1 | 2 1 0
└──────────┘
```

Patch 3x3 extrait : $[[0,2,3],[1,1,2],[1,3,1]]$

```
(1x-1)
0 1 0 1 0 2
2 1 2 0 1 1
1 0 1 2 0 0
1 2 0 1 1 1
```

Calcul : $(0 \times 1) + (2 \times -1) + (3 \times 0) + (1 \times -1) + (1 \times 2) + (2 \times 1) + (1 \times 0) + (3 \times 1) +$

$= 0 - 2 + 0 - 1 + 2 + 2 + 0 + 3 - 1 = 3$

feature_map[0, 1] = 3 (avant ReLU)

POSITION 3 (i=1, j=0) → feature_map[1, 0]

```
1 0 2 3 1 0 1
┌──────────┐
│ 0 1 1 | 2 0 1 0
│ 2 1 3 | 1 2 1 0
│ 1 0 1 | 0 1 0 2
```

Patch 3x3 extrait : $[[0,1,1],[2,1,3],[1,0,1]]$

(1x-1)

Calcul : $(0 \times 1) + (1 \times -1) + (1 \times 0) + (2 \times -1) + (1 \times 2) + (3 \times 1) + (1 \times 0) + (0 \times 1) +$

	= 0 - 1 + 0 - 2 + 2 + 3 + 0 + 0 - 1 = 1
0 2 1 2 0 1 1	
1 1 0 1 2 0 0	feature_map[1, 0] = 1 (avant ReLU)
0 1 2 0 1 1 1	

FEATURE MAP 5x5 RÉSULTANTE (partiellement remplie)

<table border="0"> <tr><td>2</td><td>3</td><td>?</td><td>?</td><td>?</td></tr> <tr><td>1</td><td>?</td><td>?</td><td>?</td><td>?</td></tr> <tr><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td></tr> <tr><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td></tr> <tr><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td></tr> </table>	2	3	?	?	?	1	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	← ligne 0 (positions j=0..4) ← ligne 1 ← lignes 2-4 calculées de la même façon
2	3	?	?	?																						
1	?	?	?	?																						
?	?	?	?	?																						
?	?	?	?	?																						
?	?	?	?	?																						

25 positions au total (5x5), une valeur calculée par position.

Ensuite ReLU : $\max(\text{valeur}, 0)$ → active les détections positives seulement.

BILAN :

Image 7x7 → 1 kernel 3x3 → 1 feature map 5x5

32 kernels en parallèle → 32 feature maps 5x5 → volume 5x5x32

Chaque kernel = un détecteur de motif différent (bords, coins, textures...)

Quand on applique un kernel à une image entière, on obtient une [feature map](#) (voir plus sur le [feature mapping](#)): une nouvelle image qui encode la présence du motif détecté par ce kernel, position par position.

Un CNN apprend des dizaines, voire des centaines de kernels différents en parallèle. Les premières couches apprennent des détecteurs simples (bords, coins, gradients de couleur). Les couches suivantes combinent ces détecteurs pour reconnaître des formes plus complexes (yeux, oreilles, museaux). C'est une hiérarchie de représentations qui émerge automatiquement de l'entraînement.

Convolution ou corrélation croisée ? Au sens mathématique strict, ce qu'on vient de calculer est une *corrélation croisée* : une vraie convolution retournerait le kernel avant de l'appliquer. En deep learning, tout le monde dit "convolution" pour les deux, et ça ne change rien en pratique : le réseau apprend ses poids de toute façon, retournés ou non, il tombe sur les bons.

Creusons

L'héritage de la vision par ordinateur classique

Avant le deep learning, les ingénieurs en vision définissaient leurs filtres à la main. Le [filtre de Sobel](#) détecte les bords horizontaux et verticaux. Le [filtre gaussien](#) floute l'image pour réduire le bruit. [Canny](#) combine plusieurs étapes pour extraire les contours.

Ces filtres sont fixes, conçus par des experts. Un CNN apprend des filtres analogues, mais adaptés aux données et à la tâche, sans intervention humaine. C'est ça, le "feature engineering automatique" qu'on invoque souvent dans la littérature DL.

La bibliothèque [OpenCV](#) contient des centaines de ces filtres classiques. Si vous avez un stage en vision, vous en croiserez inévitablement. Le DL ne les a pas remplacés dans tous les contextes : pour des pipelines industriels simples (détection de défauts sur une surface uniforme), un filtre de Sobel + seuillage bat souvent un CNN en coût de calcul et en explicabilité.

3 - Architecture CNN : shapes étape par étape

L'essentiel

On va construire un CNN comme on construirait une recette. Et avant de mettre le plat au four, on veut savoir exactement ce qu'il se passe à chaque étape.

Voilà les briques qu'on empile ce matin, dans l'ordre :

- la **convolution** (le kernel qui glisse, qu'on vient de voir) : elle transforme une image en **feature map** ;
- le **padding** et le **stride** : les deux réglages qui décident de la taille de cette feature map ;
- le **pooling** : réduire la taille spatiale sans rien apprendre ;
- le **Flatten** (ou le Global Average Pooling) : le pont entre la partie convolutive et les couches Dense finales.

À la fin, on assemble tout dans un seul tableau de shapes, en calculant chaque dimension à la main avant de laisser Keras le faire à notre place. Chaque sous-partie qui suit correspond à une de ces briques.

Si on applique un kernel 3x3 sur une image 7x7 sans padding, quelle est la taille de la feature map en sortie ?

La réponse n'est pas 7. C'est 5. Et on peut le calculer avec une formule.

La formule de la dimension de sortie

Pour une dimension (hauteur ou largeur) :

$$W_{out} = \text{floor} ((W_{in} - K + 2 * P) / S) + 1$$

Ou :

- W_{in} : taille de l'image en entrée
- K : taille du kernel (3 pour un kernel 3x3)
- P : padding (nombre de pixels zero ajoutés autour de l'image)
- S : stride (c'est le "pas" du kernel -> de combien de cases on décale le kernel à chaque étape)

Exemple : 7x7, kernel 3x3, padding=0, stride=1 : $\text{floor}((7 - 3 + 0) / 1) + 1 = 5$. La feature map fait 5x5.

Cette formule, c'est ce que Keras calcule pour vous. Mais si vous la comprenez, vous pouvez anticiper chaque dimension sans exécuter le code.

Padding : same vs valid

`padding='valid'` : pas de padding. La feature map est plus petite que l'image. Avec un kernel 3x3, chaque couche réduit la dimension de 2 (1 pixel de chaque côté). Sur 5 couches, une image 32x32 devient 22x22. Sur 10 couches, elle disparaît.

padding='same' : on ajoute des zeros autour de l'image pour que la feature map conserve la même taille spatiale que l'entrée. Avec stride=1, la formule donne exactement W_in. C'est le choix le plus fréquent dans les architectures modernes.

PADDING ET STRIDE : COMPARAISON SUR IMAGE 5x5, KERNEL 3x3

CAS A : padding='same', stride=1

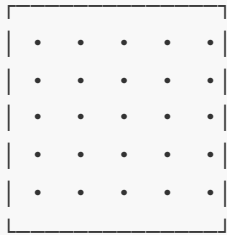
CAS B : padding='valid', stride=2

Image 5x5 avec padding=1 ajouté :

```
0 0 0 0 0 0 0
0 [1 2 3 4 5] 0
0 [6 7 8 9 0] 0
0 [1 2 3 4 5] 0
0 [6 7 8 9 0] 0
0 [1 2 3 4 5] 0
0 0 0 0 0 0 0
```

Kernel se déplace de 1 en 1 :
positions j = 0, 1, 2, 3, 4
positions i = 0, 1, 2, 3, 4

Formule :
 $W_{out} = (5 - 3 + 2 \times 1) / 1 + 1 = 5$



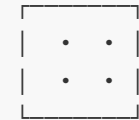
Feature map : 5x5
(même taille que l'entrée !)

Image 5x5 sans padding :

```
1 2 3 4 5
6 7 8 9 0
1 2 3 4 5
6 7 8 9 0
1 2 3 4 5
```

Kernel se déplace de 2 en 2 :
positions j = 0, 2 (seulement)
positions i = 0, 2 (seulement)

Formule :
 $W_{out} = \text{floor}((5 - 3 + 0) / 2) + 1$
 $= \text{floor}(2 / 2) + 1 = 2$



Feature map : 2x2

(division spatiale par stride)

TABLEAU RÉCAPITULATIF (kernel 3x3 sur image 5x5) :

Config	Padding	Stride	Feature map out
CAS A (same s=1)	P=1	S=1	5x5 (inchangé)
CAS B (valid s=2)	P=0	S=2	2x2 (réduit)
valid, s=1	P=0	S=1	3x3 (réduit)
same, s=2	P=1	S=2	3x3 (réduit)

RÈGLES À RETENIR :

padding='same' → la taille spatiale est conservée (si stride=1)
padding='valid' → la taille diminue de $(K-1) = 2$ pixels par couche
stride > 1 → divise la taille spatiale (approximativement par stride)

Avant : 3 couches Conv avec padding='valid' sur une image 32x32.

32x32 → Conv1 (3x3) → 30x30 → Conv2 (3x3) → 28x28 → Conv3 (3x3) → 26x26

Après : 3 couches Conv avec padding='same'.

32x32 → Conv1 (3x3) → 32x32 → Conv2 (3x3) → 32x32 → Conv3 (3x3) → 32x32

Un réseau avec padding valid et beaucoup de couches se "consomme" lui-même. Keras ne plante pas silencieusement : si la dimension devient 0 ou negative, vous obtenez une erreur.

Stride : la réduction spatiale volontaire

Le stride, c'est le pas du kernel. Avec stride=1, il se déplace d'un pixel à la fois. Avec stride=2, il saute deux pixels.

Stride=2 divise la dimension spatiale par 2 (approximativement). C'est une façon de réduire la résolution sans pooling. Les architectures modernes (ResNet, MobileNet) utilisent souvent stride=2 dans certaines couches plutôt qu'un pooling séparé.

Edge case à connaître : si $(W_{in} - K + 2P)$ n'est pas divisible par S, le résultat n'est pas entier. Keras tronque (floor), mais Keras peut aussi planter selon la configuration. Si vous voyez une erreur "invalid input shape" avec un stride inhabituel, c'est souvent ça.

Pooling : réduire sans apprendre

Le **Max Pooling** applique une fenêtre (généralement 2x2) sur la feature map et garde le maximum de chaque patch. Résultat : la dimension spatiale est divisée par 2, mais le nombre de feature maps reste constant.

Pourquoi le maximum ? Parce qu'on veut garder le "signal le plus fort" dans chaque zone. Si un détecteur de bord s'active fortement quelque part dans le patch, on veut garder cette information, même si on ne sait plus exactement où dans le patch.

L'**Average Pooling** fait la moyenne au lieu du maximum. Il est moins courant en classification mais utile dans des architectures spécifiques (comme la couche finale en transfer learning, on y revient).

Le **Global Average Pooling** (GAP) est une variante particulière : au lieu d'une fenêtre 2x2, il prend la moyenne de TOUTE la feature map. Si vous avez 512 feature maps de taille 7x7, le GAP produit un vecteur de 512 valeurs. C'est une façon élégante de passer du volume 3D à un vecteur 1D sans couche Flatten, et ça réduit drastiquement le nombre de paramètres. On l'utilisera en TP3 avec MobileNetV2.

La couche Flatten

Le Flattening est une technique qui va permettre de convertir des tableaux multidimensionnels en un tableau unidimensionnel (1D). Généralement on l'emploie en DL pour fournir les infos contenues dans ce tableau unidimensionnel au modèle de classification

Grâce à ça, par exemple on transforme un volume 3D (H x W x C) en un vecteur 1D pour les couches Dense. C'est le "pont" entre la partie convolutive du réseau et la tête de classification.

Idem si on voulait transformer une matrice 2D en un seul vecteur ligne 1D.

Pourquoi on fait ça ? on part forcément d'un CNN, cela veut dire qu'on a nécessairement des convolutions produisant des cartes en 2D/3D, mais les couches finales de décisions (**couches denses**) attendent un vecteur plat. Flatten fait le pont entre les deux (c'est le traducteur) : [convolutions] -> cartes 2D -> FLATTEN -> vecteur 1D -> [couches denses]

Si votre dernière couche convolutive sort un volume 4x4x512, Flatten produit un vecteur de $4 \times 4 \times 512 = 8192$ éléments. Ces 8192 éléments sont ensuite connectés aux couches Dense comme en J1-J2.

Le GAP remplace Flatten car il fait la même transition 3D->1D mais différemment : au lieu de dérouler tous les pixels en un vecteur géant, il calcule une seule moyenne par feature map. On a donc 512 feature maps de 7x7 donnant 512 valeurs via GAP, au lieu de $512 \times 7 \times 7 = 25088$ valeurs via Flatten.

Concrètement, trois gains :

- beaucoup moins de paramètres dans la couche Dense qui suit ;
- une régularisation implicite : la moyenne agrège l'info spatiale, donc le modèle ne peut pas sur-apprendre sur des positions précises ;
- une compatibilité avec des images de tailles variables (Flatten, lui, exige une taille fixe).

Tableau de shapes

Si on construit un tableau et qu'on calcule la case "Shape sortie"

Image d'entrée : 128x128x3 (une image couleur de 128 pixels de côté).

Couche	Paramètres	Shape sortie
Input	-	128 x 128 x 3
Conv2D(32, 3x3, padding='same')	32 filtres, kernel 3x3x3	128 x 128 x 32
MaxPooling2D(2x2)	-	64 x 64 x 32
Conv2D(64, 3x3, padding='same')	64 filtres, kernel 3x3x32	64 x 64 x 64
MaxPooling2D(2x2)	-	32 x 32 x 64
Conv2D(128, 3x3, padding='same')	128 filtres, kernel 3x3x64	32 x 32 x 128
MaxPooling2D(2x2)	-	16 x 16 x 128
Flatten	-	32 768

Couche	Paramètres	Shape sortie
Dense(128, ReLU)	-	128
Dense(1, sigmoid)	-	1 (probabilité chat vs chien)

FLOW DE SHAPES : CNN CLASSIFICATION (image 128x128x3)

Input → Conv+Pool x3 → Flatten → Dense → Dense

INPUT
128 x 128 x 3
(image couleur)

Shape : (batch, 128, 128, 3)
3 canaux couleur (RGB)



Conv2D(32, 3x3)
padding='same'
activation='relu'

Params : $3 \times 3 \times 3 \times 32 + 32 = 896$
Chaque filtre détecte un motif différent



Shape : (batch, 128, 128, 32) ← même taille spatiale (same)
32 feature maps parallèles

MaxPooling2D(2x2)
(pas de paramètres)

Prend le max sur chaque fenêtre 2x2
Divise la taille spatiale par 2



Shape : (batch, 64, 64, 32) ← $128/2 = 64$

Conv2D(64, 3x3)
padding='same'
activation='relu'

Params : $3 \times 3 \times 32 \times 64 + 64 = 18\,496$
64 détecteurs de motifs plus complexes
(basés sur les 32 feature maps précédentes)



Shape : (batch, 64, 64, 64) ← même taille spatiale

MaxPooling2D(2x2)

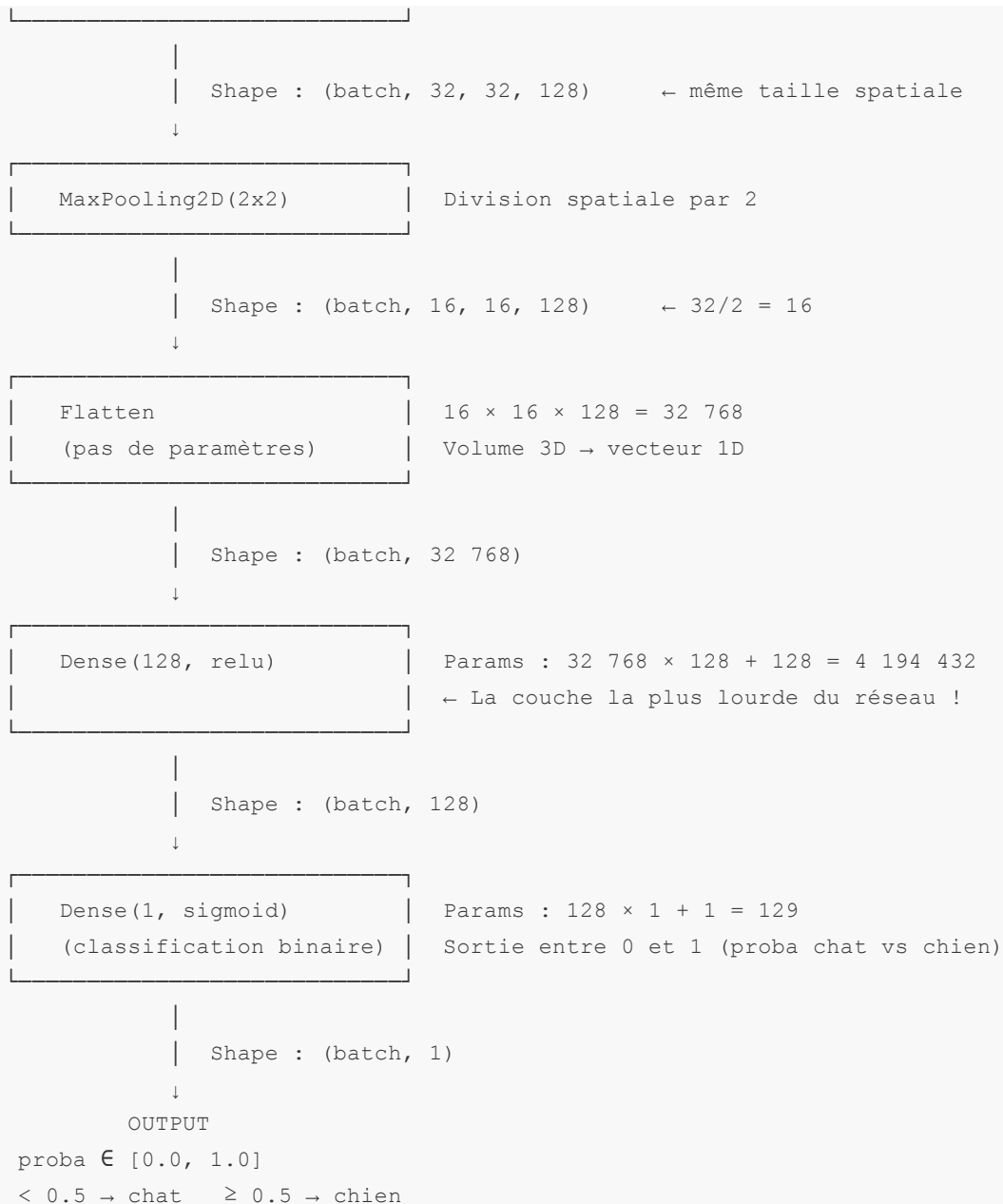
Division spatiale par 2



Shape : (batch, 32, 32, 64) ← $64/2 = 32$

Conv2D(128, 3x3)
padding='same'
activation='relu'

Params : $3 \times 3 \times 64 \times 128 + 128 = 73\,856$
128 détecteurs de concepts abstraits
(formes, structures, parties d'objets)



BILAN DES PARAMÈTRES :

Conv2D(32)	:	896	params	(presque rien)
Conv2D(64)	:	18 496	params	
Conv2D(128)	:	73 856	params	
Dense(128)	:	4 194 432	params	← 97% du total !
Dense(1)	:	129	params	
TOTAL	:	4 287 809	params	(~4.3M)

Le nombre de paramètres par couche Conv2D : pour Conv2D(32, 3x3) sur une entrée à 3 canaux : $(K \times K \times C_{in}) \times C_{out} + C_{out}$ (biais) = $(3 \times 3 \times 3) \times 32 + 32 = 896$ paramètres. C'est négligeable comparé aux millions de poids d'un PMC flatten.

Pourquoi est-ce qu'on augmente le nombre de filtres ($32 \rightarrow 64 \rightarrow 128$) à mesure qu'on descend dans le réseau, alors que les feature maps deviennent de plus en plus petites ?

C'est parce que les couches profondes détectent des concepts de plus en plus abstraits et complexes. Il faut plus de "détecteurs spécialisés" pour capturer cette diversité. En même temps, la réduction spatiale compense : le volume total de données reste à peu près constant d'une couche à l'autre.

Pour aller plus loin

Le receptive field

Un neurone dans la troisième couche convolutive "voit" quelle région de l'image d'origine ?

Avec un kernel 3x3 et stride=1 : chaque couche ajoute 1 pixel de chaque côté au champ réceptif (receptive field). Après la couche 1 : chaque neurone voit un patch 3x3 de l'image originale. Après la couche 2 : 5x5. Après la couche 3 : 7x7.

Avec du pooling (stride=2 tous les deux couches), le receptive field grandit géométriquement. Un neurone dans les couches profondes d'un ResNet peut avoir un receptive field qui couvre l'image entière.

Convolutions dilatées (pour la culture)

La [convolution dilatée](#) (dilated convolution) agrandit le receptive field sans ajouter un seul paramètre. L'idée : au lieu de prendre les pixels adjacents, le kernel "saute" des pixels selon un facteur de dilatation, donc il couvre une zone plus large avec exactement le même nombre de poids. Utile en segmentation sémantique (U-Net, DeepLab). Pas utilisé dans les architectures qu'on code aujourd'hui, mais vous en croirez en lecture de papiers.

Avantages

- Increased receptive field without increasing parameters
- Can capture features at multiple scales
- Reduced spatial resolution loss compared to regular convolutions with larger filters

Convolutions séparables depthwise (l'astuce de MobileNet)

MobileNetV2 doit son efficacité à une astuce : au lieu d'un kernel 3x3xC qui fait tout en une passe, on fait deux opérations séparées.

1. **Depthwise convolution** : un kernel 3x3 par canal d'entrée, indépendamment. Coûteux en spatial, pas en profondeur.
2. **Pointwise convolution** : un kernel 1x1 qui combine les canaux.

Le coût en paramètres passe de $K^2 * C_{in} * C_{out}$ à $K^2 * C_{in} + C_{in} * C_{out}$, soit environ 8-9 fois moins de paramètres pour un kernel 3x3. On l'utilise sans le réimplémenter en TP3 (Keras le fait), mais voilà comment ça marche.

4 - Surmonter l'overfitting image : augmentation + dropout

L'essentiel

On a 25 000 images de chats et de chiens. C'est beaucoup ? Pas vraiment.

Un CNN moderne peut avoir des millions de paramètres. Le ratio données/paramètres est mauvais par défaut. Et contrairement aux données tabulaires, chaque image est "unique" dans sa composition exacte, mais sémantiquement identique à des milliers d'autres ("un chat en intérieur, éclairage chaud, vu de face"). Le réseau risque de mémoriser les détails parasites plutôt que les vraies caractéristiques.

Si on retourne horizontalement toutes nos images, est-ce que notre dataset a doublé ?

Oui et non. Le nombre de fichiers double, mais l'information réelle ne double pas vraiment. Par contre, le modèle n'a plus la possibilité de tricher en mémorisant "ce chat est toujours à gauche de l'image". C'est ce principe que la data augmentation exploite.

Data augmentation

La [data augmentation](#) crée de la diversité synthétique sans collecter de nouvelles images. On applique des transformations aléatoires à chaque image à chaque epoch : le modèle voit à chaque passage une version légèrement différente, jamais exactement la même.

Transformations standard (et quand les utiliser) :

- **Flip horizontal** : presque toujours utile pour les photos naturelles. Un chat retourné reste un chat.
- **Rotation (-15° à +15°)** : utile pour les photos naturelles, dangereux pour la reconnaissance de caractères (un "6" tourné à 180° devient un "9").
- **Zoom** : simuler des distances différentes à la caméra. Attention aux artefacts sur les bords.
- **Décalage (translation)** : l'objet n'est pas toujours centré. Robustifie bien.
- **Ajustement contraste/luminosité** : simule des conditions d'éclairage différentes.

Transformations à éviter :

- **Flip vertical** : rarement utile pour des photos naturelles. Un chien à l'envers reste bizarre.
- **Rotation extrême (> 30°)** : introduit trop de distorsion pour la plupart des cas d'usage.

Règle absolue : l'augmentation s'applique uniquement au train set. Jamais au val set, jamais au test set. Si votre validation set est augmenté, vous évaluez sur des données différentes de celles que vous avez annoncées, et vos métriques ne signifient plus rien.

Dans Keras 3, l'approche moderne est de définir l'augmentation comme des couches dans le modèle lui-même (pas comme un générateur externe). Ces couches sont actives en mode `training=True` et passives en mode `training=False` (inférence) :

```

from tensorflow import keras
from tensorflow.keras import layers

# Ces couches sont inactives pendant l'évaluation : la val_accuracy est calculée sur
# les images originales, pas sur des versions augmentées. C'est voulu.
data_augmentation = keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),    # facteur entre 0 et 1 : ici +/-10% de 360 deg = +/-36
deg
    layers.RandomZoom(0.1),        # +/-10% de zoom
], name="augmentation")

```

Dropout en contexte CNN

On a vu le Dropout en J2. En CNN, le placement diffère :

- **Après Flatten** (avant les couches Dense) : c'est le placement le plus fréquent. Taux standard : 0.3 à 0.5.
- **Entre les couches Dense** : fonctionne, mêmes règles qu'en PMC.
- **Dans les couches Conv** : rare dans les architectures classiques. On utilise plutôt la Batch Normalization.

Batch Normalization en CNN (pour la culture)

La [Batch Normalization](#) normalise les activations de chaque couche pendant l'entraînement (moyenne nulle, variance unitaire sur le batch). Effets : convergence plus rapide, dépendance au learning rate réduite, léger effet régularisant.

Elle se place après la couche Conv et avant l'activation ReLU dans la plupart des architectures modernes. On ne l'intègre pas dans nos TP aujourd'hui, mais vous la verrez dans tous les papiers d'architecture. Le code Keras est simple :

```

# Exemple : Conv → BatchNorm → ReLU (ordre standard dans ResNet).
# On ne l'utilise pas en TP3 mais c'est le pattern de toutes les architectures
# post-2015.
x = layers.Conv2D(64, 3, padding='same', use_bias=False)(x)
x = layers.BatchNormalization()(x)
x = layers.Activation('relu')(x)

```

Ce qui peut casser

Augmentation appliquée au val/test set. Si on map les RandomFlip/RandomRotation sur le val_ds aussi, on évalue sur des images différentes à chaque epoch. La val_accuracy fluctue artificiellement et n'est plus comparable d'un run à l'autre. Reflex : **l'augmentation ne va QUE dans le pipeline du train_ds, ou dans des couches du modèle actives uniquement en `training=True`.**

Mais la différence ? [Training vs Testing vs Validation Sets](#)

Dropout trop agressif (> 0.5) sur CNN. Le train_accuracy stagne sous 70%, la val_accuracy idem. Le réseau n'apprend plus parce qu'on coupe trop de signal à chaque forward. Sur les blocs conv, 0.2-0.3 est déjà le plafond utile. Au-delà, c'est du sous-apprentissage induit.

Augmentation incohérente avec la tâche. Flip vertical sur des photos naturelles (un chien à l'envers, ça n'existe pas), rotation 180 sur de l'OCR (un 6 devient un 9), miroir sur du texte. Le modèle apprend des règles qui n'existent pas dans le test set. Reflex : visualiser 10 augmentations avant d'entraîner, vérifier que chaque image reste plausible.

Batch Normalization avec batch_size < 16. Les statistiques de moyenne et de variance calculées sur chaque batch deviennent bruitées quand le batch est trop petit. Résultat : les courbes de loss oscillent au lieu de descendre proprement, la convergence part dans tous les sens. Sur petit batch, préférez GroupNorm ou montez le batch_size, sinon retirez la BN.

Approfondissons

L'analogie pédagogique

L'augmentation, c'est comme faire travailler un étudiant sur le même exercice avec les variables permutées. Celui qui a toujours vu " $x + 2 = 5$, trouver x " peut résoudre ça en mémorisant. Celui qui a vu " $a + 7 = 12$ ", " $b + 3 = 8$ ", " $c - 4 = 1$ " doit comprendre le principe. Il généralise mieux.

Le **CNN avec augmentation est l'étudiant qui généralise**. Celui **sans augmentation est celui qui mémorise**.

5 - Transfer learning et architectures de référence

L'essentiel

Imaginez que vous devez apprendre à reconnaître des chats.

Option 1 : partez de zéro, comme un nouveau-né. Apprenez ce qu'est une ligne, puis une courbe, puis une forme, puis une tête, puis une oreille pointue, puis un chat. Des semaines d'expérience visuelle.

Option 2 : vous avez déjà appris à voir des contours, des textures, des formes, des objets en 3D, pendant des semaines sur 14 millions d'images. Il ne vous reste qu'à ajuster les dernières couches pour différencier chats et chiens. C'est ça, le **transfer learning**.

ImageNet : le dataset de référence

[ImageNet](#) est la base de données qui a révolutionné la vision par ordinateur. 14 millions d'images annotées, 1000 classes (de "goldfish" à "container ship"), organisées selon la hiérarchie lexicale WordNet.

Le [Large Scale Visual Recognition Challenge](#) (ILSVRC, 2010-2017) a poussé les chercheurs à concevoir des architectures de plus en plus efficaces pour ce benchmark. AlexNet (2012), VGG (2014), Inception (2014), ResNet (2015), MobileNet (2017) : toutes ces architectures ont été entraînées sur ImageNet.

Les poids pré-entraînés sur ImageNet ont déjà appris des représentations générales. Les premières couches détectent des bords et des gradients (universels, indépendants du domaine). Les couches intermédiaires détectent des textures et des formes. Les couches profondes détectent des concepts sémantiques (yeux, roues, feuilles). Ces représentations **transfèrent bien** même sur des domaines éloignés d'ImageNet.



Pourquoi ça marche sur des chats et des chiens alors qu'ImageNet contient déjà des chats et des chiens ? Parce que même sur un domaine complètement différent (imagerie médicale, photos satellites, radiographies), les représentations de bas niveau (bords, textures) restent utiles. Le transfer learning fonctionne sur des domaines où ImageNet n'a jamais vu une seule image du domaine cible.

Architectures de référence

Six noms qui reviennent tout le temps. Bonne nouvelle : vous n'avez pas à tous les retenir. Deux comptent vraiment pour vous aujourd'hui : **MobileNetV2** (celui du TP de cet après-midi) et **ResNet** (le standard qu'on croise partout en entreprise). Les quatre autres, c'est de la culture pour situer une archi le jour où vous la croisez dans un papier ou un repo. On les prend dans l'ordre historique, parce que chacune a résolu un problème que la précédente laissait ouvert.

LeNet (1998, LeCun et al.) : le premier CNN viable. 2 couches Conv + pooling, 3 couches Dense. Conçu pour reconnaître des chiffres manuscrits en 32x32 pixels en niveaux de gris. 60 000 paramètres. Pas utilisé en prod aujourd'hui, mais fondamental pour comprendre la progression historique : le même principe, empilé et raffiné pendant 20 ans.

VGG16 / VGG19 (2014, Simonyan et Zisserman) : architecture simple et profonde. 16 ou 19 couches de poids, tous les kernels en 3x3, empilés en blocs. La simplification radicale (toujours 3x3) a montré qu'on pouvait battre des architectures plus complexes avec plus de profondeur et moins de magie. VGG16 = 138 millions de paramètres. Lourd, mais très lisible. Bon pour comprendre le fine-tuning, mentionné en alternative dans les phases avancées de l'après-midi.

Inception / GoogLeNet (2014, Szegedy et al.) (pour la culture) : le module inception applique plusieurs tailles de kernels (1x1, 3x3, 5x5) en parallèle sur la même entrée et concatène les résultats. L'idée : laisser le réseau choisir la "bonne" échelle à chaque couche. 22 couches de profondeur, seulement 5 millions de paramètres grâce aux convolutions 1x1.

ResNet (2015, He et al.) : residual connections. Le problème qu'elle résout : plus on empile des couches, plus le gradient disparaît en backpropagation. La solution : ajouter un "shortcut" qui bypass 2 couches. Le gradient a un chemin direct vers les premières couches. ResNet50 (50 couches, 25M params) est un benchmark standard dans l'industrie. Si votre sujet J5 implique de la vision et que vous avez du temps, c'est une alternative solide à MobileNetV2.

MobileNet / MobileNetV2 (2017/2018, Google) : conçu pour les appareils mobiles et embarqués. Depthwise separable convolutions (vu en section 3). MobileNetV2 = 4.2 millions de paramètres, précision proche de VGG16 sur ImageNet, vitesse d'inférence 5-10x plus rapide. C'est le choix du TP3 : léger, rapide sur Colab T4, et il se convertit en TFLite pour le déploiement mobile.

EfficientNet (2019, Tan et Le) (pour les curieux) : scaling composé. Plutôt que d'augmenter la profondeur, la largeur, ou la résolution indépendamment, EfficientNet les scale tous les trois selon un rapport fixe. EfficientNetB0 à B7 : du plus léger au plus précis. État de l'art sur ImageNet pendant plusieurs années.

Stratégies de transfer learning

Trois stratégies, selon la taille du dataset et la proximité du domaine avec ImageNet :

Stratégie	Quand	Learning rate	Données requises	Modèle conseillé
Freezing total	Dataset petit, domaine proche ImageNet	Normal	Quelques centaines	MobileNetV2, EfficientNetB0
Fine-tuning partiel	Dataset moyen, domaine similaire	10x plus petit	Quelques milliers	ResNet50, MobileNetV2

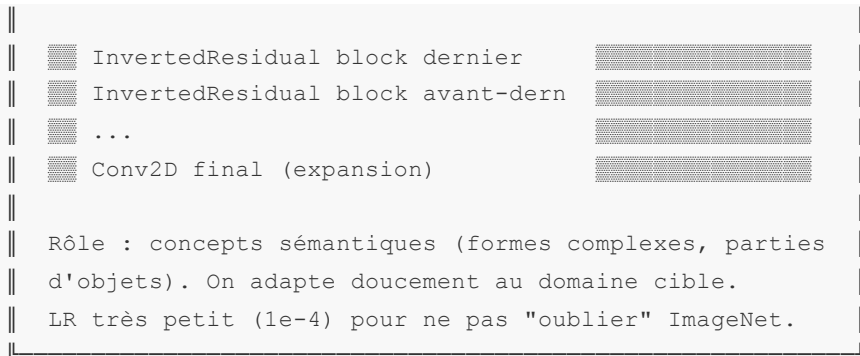
Stratégie	Quand	Learning rate	Données requises	Modèle conseillé
Fine-tuning complet	Dataset grand, domaine très différent	100x plus petit	Dizaines de milliers	EfficientNetB4-B7, ResNet101

Freezing total : `base_model.trainable = False`. Les poids de la base pré-entraînée ne se mettent pas à jour pendant l'entraînement. Seule la tête de classification (les couches qu'on ajoute au-dessus) est entraînée. Rapide, efficace avec peu de données.

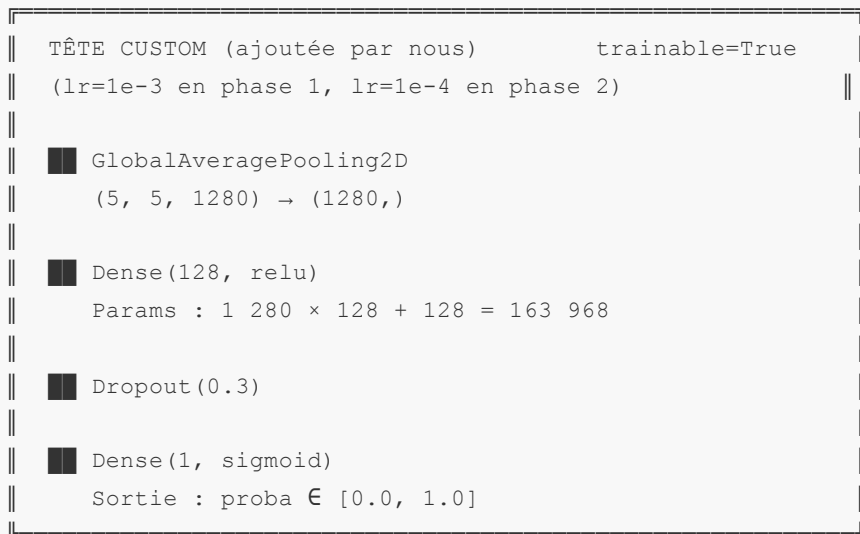
Fine-tuning partiel : on dégèle les dernières couches de la base. Ces couches encodent des représentations de haut niveau (spécifiques au domaine d'ImageNet). En les réentraînant avec un learning rate très petit, on les adapte au domaine cible sans "oublier" ce qu'elles ont appris sur ImageNet. En Keras : `base_model.trainable = True` puis `layer.trainable = False` pour les N premières couches.

Fine-tuning complet : tout le réseau est réentraînable. Risque d'oubli catastrophique si le dataset est petit. À éviter sauf si vous avez des dizaines de milliers d'images dans un domaine très différent d'ImageNet.





↓ Shape : (batch, 5, 5, 1280)



↓
OUTPUT : proba chat vs chien

COMPTE DE PARAMÈTRES PAR ZONE :

Zone gelée	: ~2 100 000 params	→ Non-trainable
Zone fine-tuning	: ~500 000 params	→ Trainable (lr=1e-4)
Tête custom	: ~164 000 params	→ Trainable (lr=1e-4)
TOTAL trainable (phase 2)	: ~664 000	(phase 1 : ~164 000 seulement)

LÉGENDE :

	Zone gelée	(trainable=False, gradient ne passe pas)
	Zone dégelée	(trainable=True, fine-tuning lr=1e-4)
	Tête custom	(trainable=True, entraînée depuis le début)

```
import tensorflow as tf
from tensorflow.keras import layers

# input_shape doit être compatible avec MobileNetV2 : minimum 32x32, typiquement 160x160.
base_model = tf.keras.applications.MobileNetV2(
```

```

input_shape=(160, 160, 3),
include_top=False,
weights='imagenet'
)

base_model.trainable = False

inputs = tf.keras.Input(shape=(160, 160, 3))
x = base_model(inputs, training=False) # training=False : BN en mode inférence même si le
modele est en training
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(128, activation='relu')(x)
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(1, activation='sigmoid')(x)

model = tf.keras.Model(inputs, outputs)

```

Dans l'industrie, 90% des projets de vision partent d'un modèle pré-entraîné. Partir de zéro, c'est réservé aux équipes qui ont des TPU et 6 mois devant elles. L'après-midi, vous allez voir pourquoi en 15 lignes de code.

Pour les curieux

Jusqu'où faut-il dégeler ?

Il n'y a pas de règle universelle. Des heuristiques pratiques :

- Si votre val_accuracy stagne après le freezing total : dégelez les 20-30 derniers pourcents des couches et réentraînez avec `1r / 10`.
- Si après le fine-tuning partiel la val_loss re-augmente : votre learning rate est trop grand. Réduisez encore.
- Si votre dataset fait moins de 1000 images : restez en freezing total, le fine-tuning overfit à coup sûr.

La heatmap de Grad-CAM (Gradient-weighted Class Activation Mapping) permet de visualiser "où le modèle regarde" dans l'image pour prendre sa décision. Un outil utile pour diagnostiquer si le modèle a bien transféré ses représentations ou s'il overfit sur des artefacts.

6 - Comparaison Keras / PyTorch

Pour la culture : le cours reste en Keras. En J5, vous choisissez librement votre framework.

Dans votre carrière, vous rencontrerez des repos [PyTorch](#), des tutos PyTorch, des papiers avec du code PyTorch. 15 minutes pour que ce soit lisible :

Le même CNN 3 couches, côte à côte :

	Keras (TF)	PyTorch
Import	<code>from tensorflow import keras</code>	<code>import torch.nn as nn</code>
Définition	<code>keras.Sequential([...])</code>	Class qui hérite de <code>nn.Module</code> + méthode <code>forward()</code>
Couche Conv	<code>layers.Conv2D(32, 3, activation='relu')</code>	<code>nn.Conv2d(in_channels, 32, kernel_size=3) + nn.ReLU()</code>
Couche Dense	<code>layers.Dense(128, activation='relu')</code>	<code>nn.Linear(in_features, 128) + nn.ReLU()</code>

	Keras (TF)	PyTorch
Compilation	<code>model.compile(optimizer='adam', loss='bce')</code>	<code>optimizer = torch.optim.Adam(model.parameters()) + criterion = nn.BCELoss()</code>
Entraînement	<code>model.fit(X, y, epochs=10)</code>	Boucle manuelle : <code>for epoch in range(10): for batch in dataloader:...</code>
Inference	<code>model.predict(X)</code>	<code>model.eval() + with torch.no_grad(): outputs = model(X)</code>

Keras masque beaucoup de détails (boucle d'entraînement, gestion du gradient). PyTorch les expose explicitement. Avantages de chaque approche :

- Keras : a un code plus court, moins de boilerplate, excellent pour le prototypage.
- PyTorch : permet un contrôle granulaire, plus facile à déboguer pas à pas, dominant dans la recherche.

Si vous lisez un papier arXiv avec du code : c'est probablement du PyTorch. Si vous déployez une API en prod avec une contrainte de rapidité : TF/Keras avec TFLite ou TF Serving.

Let's go ! Projet de l'après-midi

Le projet

Trois itérations sur le même problème de classification binaire d'images, avec le dataset de votre choix. Quelques exemples pour s'inspirer : chats vs chiens ([Cats vs Dogs Kaggle](#)), plantes saines vs malades ([Plant Disease](#)), radiographies poumons sains vs pneumonie ([Chest X-Ray](#)), vêtements par catégorie ([Fashion-MNIST](#)), ou votre propre sujet si vous avez déjà un dataset en tête.

Critère de choix : un dataset disponible sur Kaggle ou HuggingFace, téléchargeable en moins de 10 min, avec au moins 1 000 images par classe. La structure finale attendue :

```
data/
  train/
    classe_a/      (80% des images)
    classe_b/
  val/
    classe_a/      (20% des images)
    classe_b/
```

Le livrable final : un notebook avec trois modèles entraînés, leurs courbes comparatives, et un tableau de métriques qui documente la progression entre TP1 (CNN scratch), TP2 (augmentation + Dropout) et TP3 (transfer learning MobileNetV2).

Ce que le repo doit contenir à la fin de l'après-midi :

- Le notebook avec les trois TP (phases 1.x à 3.x)
- Les courbes matplotlib (loss/accuracy pour TP1, TP2, TP3)
- Le fichier `.keras` du meilleur modèle
- Le tableau de comparaison cross-modèles complet (phase 3.4)
- Phase 3.5 : le fichier `.tflite` et/ou le résultat d'une direction d'exploration

Les commits racontent votre progression : un commit après chaque phase ou groupe de phases, message clair ("TP1 CNN scratch - overfitting visible à epoch 10"), staging explicite par fichier.

Rappel notation :

Le repo GitHub est la source de vérité pour la notation continue. Critères :

- Commits fréquents (après chaque phase terminée, pas un commit unique à la fin)
- Commits atomiques (un commit = un changement logique)
- Pas de `git add .` : staging explicite fichier par fichier
- Messages clairs et en français ou en anglais, cohérents

Pas de commit, pas de trace, pas de note.

Question rapide, parmi les personnes ayant déjà traité un dataset chat/chien, est-ce qu'il y a des choses que vous n'avez jamais faites parmi les phases ci-dessous (est-ce que vous avez déjà poussé autant un dataset de ce type) :

- Phase 1.1 : Setup Colab + organisation du dataset
 - Phase 1.2 : Preprocessing : normalisation + batching
 - Phase 1.3 : Architecture CNN from scratch
 - Phase 1.4 : Entraînement et diagnostic overfitting
 - Phase 2.1 : Pipeline d'augmentation
 - Phase 2.2 : Intégration Dropout + réentraînement
 - Phase 2.3 : Diagnostic comparatif TP1 vs TP2
 - Phase 3.1 : Chargement MobileNetV2 et freezing
 - Phase 3.2 : Entraînement de la tête uniquement
 - Phase 3.3 : Fine-tuning des dernières couches
 - Phase 3.4 : Tableau de comparaison cross-modèles
 - Phase 3.5 : Export TFLite et exploration ouverte
-

Phase 1.1 : Setup Colab + organisation du dataset

Objectif : Configurer l'environnement Colab, télécharger le dataset via l'API Kaggle, organiser les dossiers en structure `train/val` par classe, et afficher quelques images pour vérifier.

Avant de coder : Un commit "setup initial" après avoir créé le notebook et vérifié la structure du dataset.

Commencez par définir les variables de configuration spécifiques à votre dataset. Ces constantes sont le seul endroit où changer de sujet.

```
# === CONFIGURATION DU PROJET ===
# TODO : renseigner les noms de vos deux classes (en snake_case, sans espace)
CLASS_A = "???" # ex : "cat", "healthy", "normal"
CLASS_B = "???" # ex : "dog", "diseased", "pneumonia"

# TODO : chemin racine où les images brutes seront extraites
DATA_ROOT = "/content/data"
```

Setup de l'API Kaggle dans Colab. L'API Kaggle nécessite un fichier `kaggle.json` (votre token d'authentification). Téléchargez-le depuis [kaggle.com/settings](https://www.kaggle.com/settings) > API > "Create New Token".

```
pip install kaggle -q
```

```
mkdir -p ~/.kaggle
```

```
from google.colab import files
files.upload() # sélectionner kaggle.json

import os
os.makedirs(os.path.expanduser("~/kaggle"), exist_ok=True)
os.rename("kaggle.json", os.path.expanduser("~/kaggle/kaggle.json"))
os.chmod(os.path.expanduser("~/kaggle/kaggle.json"), 0o600)
```

```
# TODO : remplacer <dataset-slug> par le slug de votre dataset Kaggle
# Format competition : kaggle competitions download -c <nom-compétition> -p /content/data/
# Format dataset      : kaggle datasets download -d <utilisateur>/<dataset-slug> -p
                        /content/data/
# Puis : cd /content/data && unzip -q <fichier>.zip
```

Organisation des dossiers. Quel que soit le dataset choisi, la structure finale doit suivre le format `train/classe_a/`, `train/classe_b/`, `val/classe_a/`, `val/classe_b/`. La logique exacte dépend de comment votre dataset est livré.

```
import os, shutil, random

# TODO 1 : lister les fichiers bruts pour CLASS_A et CLASS_B
#          (adapter le chemin selon la structure livrée par Kaggle)
files_a = [] # TODO : lister les images de la classe A
files_b = [] # TODO : lister les images de la classe B

# TODO 2 : créer les dossiers train/CLASS_A, train/CLASS_B, val/CLASS_A, val/CLASS_B
#          avec os.makedirs(..., exist_ok=True)

# TODO 3 : mélanger chaque liste (random.shuffle) avec seed=42 pour la reproductibilité

# TODO 4 : split 80% train / 20% val pour chaque classe
#          et copier avec shutil.copy

# Vérification : ce bloc doit tourner sans modification
for split in ['train', 'val']:
    for cls in [CLASS_A, CLASS_B]:
        path = os.path.join(DATA_ROOT, split, cls)
        print(f"{path} : {len(os.listdir(path))} images")
```

```
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

# TODO : afficher 3 images de CLASS_A et 3 images de CLASS_B côte à côte.
# Structure : plt.subplot(2, 3, i+1), plt.imshow(), plt.title(CLASS_A ou CLASS_B)
# Vérifier que les shapes se terminent par 3 (RGB) ou 1 (niveaux de gris).
```

Devrait afficher : comptes d'images cohérents par classe (ratio 80/20 respecté) et une grille 2x3 montrant des exemples des deux classes.

Verifications :

- Happy path : au moins 500 images en train pour chaque classe, ratio 80/20 respecté, grille 2x3 affichée sans erreur, les shapes d'images se terminent toutes par 3 (RGB) ou toutes par 1 (si dataset en niveaux de gris).
- Edge case : changer le seed du `random.shuffle` (tester seed=0 vs seed=42). Le split est différent mais le ratio reste identique. Vérifier que le compte d'images par split ne change que d'une unité au plus (arrondi du 80/20).
- Adversarial : `kaggle.json` mal placé ou permissions trop ouvertes. Erreur typique : `OSError: Could not find kaggle.json` OU Your Kaggle API key is readable by other users on this system. Fix : `chmod 600 ~/.kaggle/kaggle.json` et vérifier avec `ls -la ~/.kaggle/`.

Phase 1.2 : Preprocessing : normalisation + batching

Objectif : Charger les images avec `image_dataset_from_directory`, normaliser les valeurs de pixels, configurer le batching, et afficher le premier batch pour vérifier les shapes.

Rappel notation : Commit "phase 1.2 dataset pipeline" avant de continuer.

```
import tensorflow as tf

# TODO : choisir la taille de redimensionnement et la taille de batch.
# Conseils : IMG_SIZE entre 64x64 et 160x160 selon la RAM Colab disponible.
# BATCH_SIZE entre 16 et 64 (32 est un bon défaut).
IMG_SIZE = (???, ???)
BATCH_SIZE = ???

# TODO : compléter les paramètres de image_dataset_from_directory pour train_ds.
# Paramètres à renseigner : image_size, batch_size, label_mode, shuffle, seed.
train_ds = tf.keras.utils.image_dataset_from_directory(
    os.path.join(DATA_ROOT, "train"),
    # TODO : tous les paramètres
)

# TODO : même chose pour val_ds (shuffle=False pour la reproductibilité).
val_ds = tf.keras.utils.image_dataset_from_directory(
    os.path.join(DATA_ROOT, "val"),
    # TODO : tous les paramètres
)
```

```
# TODO : appliquer Rescaling(1./255) sur train_ds et val_ds avec .map().
# normalization_layer = tf.keras.layers.Rescaling(1./255)

# TODO : appliquer .cache().prefetch(buffer_size=AUTOTUNE) sur train_ds.
# et .cache().prefetch(buffer_size=AUTOTUNE) sur val_ds.
AUTOTUNE = tf.data.AUTOTUNE

# TODO : afficher la shape du premier batch (images et labels).
# Indice : next(iter(train_ds)) retourne un tuple (images, labels).
# Shape attendue images : (BATCH_SIZE, IMG_SIZE[0], IMG_SIZE[1], 3)
# Shape attendue labels : (BATCH_SIZE, 1)
# Valeurs attendues : entre 0.0 et 1.0 après Rescaling
```

Devrait afficher : `Shape images batch : (32, H, W, 3)` avec H et W correspondant à votre `IMG_SIZE`, valeurs min = 0.00, max = 1.00 (ou proche).

Trois cas à valider :

- Happy path : shape `(BATCH_SIZE, H, W, 3)` pour les images, `(BATCH_SIZE, 1)` pour les labels, valeurs entre 0.0 et 1.0 après Rescaling, aucune erreur de path.
- Edge case : oublier le `.cache()` sur `train_ds`. Le pipeline relit le disque à chaque epoch, l'entraînement devient 3-5x plus lent. Visible sur le temps par epoch dans la sortie de `model.fit()`.
- Adversarial : `label_mode='binary'` avec une structure de dossiers contenant 3 sous-dossiers (un dossier parasite créé par erreur, ex. `.ipynb_checkpoints`). Keras range alphabétiquement et lève une `ValueError` si plus de 2 classes. Reflex : `os.listdir(DATA_ROOT + "/train")` avant de lancer `image_dataset_from_directory`.

Phase 1.3 : Architecture CNN from scratch

Objectif : Construire et compiler un CNN from scratch, vérifier que les shapes du `model.summary()` correspondent à ceux calculés le matin, et identifier le nombre de paramètres.

Avant de coder : Commit "phase 1.3 architecture CNN".

```
from tensorflow.keras import layers, models

def build_cnn_scratch(input_shape):
    """
    CNN from scratch pour la classification binaire.
    Architecture délibérément simple : on veut voir l'overfitting, pas l'éviter.

    input_shape : tuple (H, W, C) correspondant à votre IMG_SIZE + nombre de canaux
    Retourne : un model Sequential non compilé
    """
    model = models.Sequential([
        # TODO : bloc 1 - Conv2D(32, (3,3), activation='relu', padding='same',
        input_shape=input_shape)
        # suivi MaxPooling2D((2,2))
        # Shape en sortie : (H/2, W/2, 32) - calculer avec votre IMG_SIZE
```

```

# TODO : bloc 2 - Conv2D(64, (3,3)) + MaxPooling2D((2,2))
# Shape en sortie : (H/4, W/4, 64) - calculer avant d'exécuter

# TODO : bloc 3 - Conv2D(128, (3,3)) + MaxPooling2D((2,2))
# Shape en sortie : (H/8, W/8, 128) - calculer avant d'exécuter

# TODO : Flatten()
# Shape en sortie : (H/8 * W/8 * 128,) - calculer le nombre total d'éléments

# TODO : Dense(128, activation='relu')

layers.Dense(1, activation='sigmoid')
])
return model

model_scratch = build_cnn_scratch(input_shape=(IMG_SIZE[0], IMG_SIZE[1], 3))
model_scratch.summary()

# TODO : compiler avec optimizer='adam', loss='binary_crossentropy', metrics=['accuracy']

```

Devrait afficher : un `model.summary()` avec les shapes que vous avez calculées à la main le matin. Point de contrôle : la shape après `Flatten` doit correspondre à votre calcul avant d'exécuter.

Questions avant d'exécuter :

- Calculez la shape en sortie du bloc 2 (après MaxPooling) pour votre IMG_SIZE choisie.
- Calculez le nombre total de paramètres de la couche `Dense(128)` à partir de la shape Flatten.

Quality gate :

- Happy path : `model.summary()` affiche 3 blocs Conv2D + Flatten + 2 Dense, output shape finale `(None, 1)`, le total params correspond à votre calcul préalable.
- Edge case : changer `padding='same'` en `padding='valid'` sur les Conv2D. Les shapes intermédiaires se réduisent (chaque Conv perd 2 pixels par bord), le Flatten produit moins d'éléments, Dense(128) a moins de paramètres. Recalculer et comparer.
- Adversarial : oublier `activation='sigmoid'` sur la dernière Dense(1). Le modèle compile sans erreur mais la `binary_crossentropy` reçoit des logits non bornés, l'entraînement diverge ou plafonne à 50%. Reflex : toujours vérifier la dernière activation avant de lancer fit().

Phase 1.4 : Entraînement et diagnostic overfitting

Objectif : Entraîner le CNN from scratch pendant 20 epochs, lancer TensorBoard, lire les courbes train/val, et identifier le point de divergence (overfitting).

Avant de lancer : Commit "phase 1.4 training scratch".

```

import datetime, time

# Callback TensorBoard : un log par run, avec un timestamp pour ne pas écraser les
précédents.
log_dir = "logs/scratch/" + datetime.datetime.now().strftime("%Y%m%d-%H%M%S")

```



```
# TODO : créer le callback TensorBoard qui log dans log_dir.
# tf.keras.callbacks.TensorBoard(log_dir=log_dir, histogram_freq=1)
tensorboard_callback = # TODO

# TODO : créer un callback EarlyStopping qui surveille 'val_loss',
# avec patience=5 et restore_best_weights=True.
early_stopping = # TODO

start = time.time()

# TODO : lancer model_scratch.fit(train_ds, epochs=20, validation_data=val_ds,
#                                 callbacks=[tensorboard_callback, early_stopping])
history_scratch = # TODO

training_time_scratch = time.time() - start
print(f"Temps d'entraînement : {training_time_scratch:.0f}s")
print(f"val_accuracy finale : {max(history_scratch.history['val_accuracy']):.3f}")
```

```
%load_ext tensorboard
%tensorboard --logdir logs/scratch
```

La fonction `plot_history` est fournie : un utilitaire de visualisation à réutiliser pour toutes les phases.

```
import matplotlib.pyplot as plt

def plot_history(history, title=""):
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

    ax1.plot(history.history['loss'], label='Train loss')
    ax1.plot(history.history['val_loss'], label='Val loss')
    ax1.set_title(f'{title} - Loss')
    ax1.legend()

    ax2.plot(history.history['accuracy'], label='Train accuracy')
    ax2.plot(history.history['val_accuracy'], label='Val accuracy')
    ax2.set_title(f'{title} - Accuracy')
    ax2.legend()

    plt.tight_layout()
    plt.savefig(f"curves_{title.lower().replace(' ', '_')}.png", dpi=100)
    plt.show()

plot_history(history_scratch, "CNN scratch")
```

Ce que vous devez observer :

La val_accuracy va plafonner pendant que la train_accuracy continue de monter. La val_loss va remonter après quelques epochs. C'est l'overfitting classique. C'est voulu, pas un échec de votre code : le CNN mémorise le train set sans généraliser. Identifiez sur les courbes : à quel epoch est-ce que la val_loss commence à remonter ? C'est le point optimal que l'EarlyStopping doit attraper.

Avant de passer à la suite :

- Happy path : train_accuracy monte vers 80-90%, val_accuracy plafonne en dessous, gap visible dès epoch 8-12, val_loss qui remonte. TensorBoard ouvert et synchrone avec les logs.
- Edge case : lancer avec `BATCH_SIZE=4` au lieu de 32. Le bruit du gradient explose, les courbes oscillent violemment, la val_accuracy finale est plus basse. Mesurer l'écart de val_accuracy finale entre les deux batch sizes.
- Adversarial : oublier `shuffle=True` dans `image_dataset_from_directory`. Les batches contiennent des images d'une seule classe d'affilée. La loss oscille epoch par epoch parce que chaque batch est quasi-pur, le modèle n'apprend rien de stable.

Phase 2.1 : Pipeline d'augmentation

Objectif : Intégrer les couches d'augmentation Keras dans le modèle, vérifier visuellement les transformations sur une grille d'images augmentées avant l'entraînement.

Rappel git : Commit "phase 2.1 data augmentation pipeline".

```
from tensorflow.keras import layers, models

# Ces couches sont actives uniquement en mode training (model.fit) et
# passives en mode inférence (model.predict, model.evaluate).
# Conséquence : la val_accuracy est calculée sur les images ORIGINALES. C'est voulu.
data_augmentation = models.Sequential([
    layers.RandomFlip("horizontal"),
    # TODO : ajouter RandomRotation avec un facteur adapté à votre domaine.
    # Indice : factor entre 0.05 (légère rotation, ~18°) et 0.15 (~54°).
    # Pour des radiographies, une rotation forte peut changer le diagnostic : choisir <
    0.1.

    # TODO : ajouter RandomZoom avec un facteur de votre choix.
    # Indice : factor entre 0.05 et 0.15.
], name="data_augmentation")

# TODO : récupérer une image d'exemple depuis train_ds (next(iter(train_ds))[0][0])
# et afficher une grille 3x3 de versions augmentées de cette même image.
# Structure :
# for i in range(9):
#     augmented = data_augmentation(tf.expand_dims(sample_image, 0), training=True)
#     plt.subplot(3, 3, i+1) ; plt.imshow(augmented[0]) ; plt.axis('off')
# Sauvegarder avec plt.savefig("augmentation_grid.png", dpi=100)
```

Devrait afficher : une grille 3x3 où la même image apparaît avec des variations aléatoires (certaines retournées, certaines tournées ou zoomées). Toutes doivent rester reconnaissables : si une transformation rend l'image ambiguë pour vous, réduire son intensité.

Checkpoint :

- Happy path : 9 versions augmentées de la même image affichées, toutes différentes (flip, rotation, zoom visibles), toutes reconnaissables comme appartenant à la même classe.

- Edge case : empiler 5 `RandomRotation(0.5)` d'affilée dans le pipeline. L'image devient méconnaissable. La data augmentation a une limite : plus n'est pas mieux. Réduire le facteur ou retirer des couches jusqu'à obtenir des transformations réalistes.
- Adversarial : appliquer `RandomFlip("vertical")` sur des photos de sujets naturels (animaux, plantes, personnes). Visualiser le résultat. Le modèle apprendrait des règles qui n'existent pas dans le test set. Sur photos naturelles orientées (tête en haut), le flip vertical est presque toujours contre-productif.

Phase 2.2 : Intégration Dropout + réentraînement

Objectif : Construire un CNN augmenté avec Dropout, l'entraîner depuis zéro sur le même dataset, comparer les courbes directement avec celles de la phase 1.4.

Avant de modifier : Commit "phase 2.2 dropout retraining".

```
def build_cnn_augmented(input_shape):
    """
    CNN avec data augmentation intégrée + Dropout.
    Même architecture de base que build_cnn_scratch, mais avec régularisation.

    input_shape : tuple (H, W, C)
    """
    model = models.Sequential([
        data_augmentation,

        # TODO : reprendre les 3 blocs Conv2D + MaxPooling de build_cnn_scratch.
        # Shape attendue identique à TP1, la data_augmentation ne change pas les shapes.

        # TODO : Flatten()

        # TODO : Dropout(0.4)
        # Coupe 40% des connexions aléatoirement à chaque batch.
        # Placer APRES Flatten (pas entre les blocs Conv) : les feature maps spatiales
        # ne doivent pas être coupées, seulement les activations après aplatissement.

        # TODO : Dense(128, activation='relu')

        layers.Dense(1, activation='sigmoid')
    ])
    return model

model_augmented = build_cnn_augmented(input_shape=(IMG_SIZE[0], IMG_SIZE[1], 3))

# TODO : compiler model_augmented (mêmes paramètres que TP1)

# TODO : créer log_dir_aug, tensorboard_callback_aug, early_stopping_aug
#         (même structure que phase 1.4, log dans "logs/augmented/...")

# TODO : lancer model_augmented.fit(...) avec les mêmes hyperparamètres que TP1
#         Stocker dans history_augmented, mesurer training_time_augmented

print(f"Temps d'entraînement : {training_time_augmented:.0f}s")
```

```
print(f"val_accuracy finale : {max(history_augmented.history['val_accuracy']):.3f}")

plot_history(history_augmented, "CNN augmenté + Dropout")
```

Ce que vous devez observer : La val_accuracy monte plus haut que TP1. La divergence train/val est réduite. La convergence est peut-être plus lente (le modèle ne peut plus mémoriser aussi facilement). C'est le signe que la régularisation fonctionne.

Verifications avant TP suivant :

- Happy path : val_accuracy supérieure à TP1, gap train/val réduit (overfit atténué mais pas nul), val_loss continue à descendre après epoch 15, EarlyStopping se déclenche plus tard qu'en TP1.
- Edge case : déplacer Dropout AVANT Flatten au lieu d'après. Les feature maps sont coupées à 40%, le modèle perd des positions spatiales entières. La val_accuracy chute, comparer avec la version Dropout post-Flatten.
- Adversarial : passer Dropout(0.8). La train_accuracy s'effondre sous 70% et la val_accuracy avec. Le modèle ne peut plus apprendre, le signal est trop bruité. Le bon range pour CNN est 0.2-0.4 après Flatten.

Phase 2.3 : Diagnostic comparatif TP1 vs TP2

Objectif : Comparer systématiquement les courbes des deux modèles, mettre les chiffres en regard, et rédiger un paragraphe d'interprétation.

Avant de comparer : Commit "phase 2.3 comparaison scratch vs augmenté".

```
# TODO : tracer val_loss de history_scratch et history_augmented sur le même graphe
(subplot 1/2).
# TODO : tracer val_accuracy de history_scratch et history_augmented sur le même graphe
(subplot 2/2).
# Structure attendue :
#   plt.subplot(1, 2, 1) pour la val_loss, plt.subplot(1, 2, 2) pour la val_accuracy
#   couleur rouge pour scratch, bleu pour augmenté
#   sauvegarder avec plt.savefig("comparison_tp1_tp2.png", dpi=100)

# TODO : afficher les métriques clés de comparaison.
# Clés à extraire :
#   - val_accuracy max pour chaque modèle (max(history.history['val_accuracy']))
#   - training_time pour chaque modèle
#   - model.count_params() pour chaque modèle
```

Dans une cellule Markdown, écrivez un paragraphe (5-8 lignes) qui répond à ces questions :

- Qu'est-ce qui a changé entre TP1 et TP2 dans les courbes ?
- Le gap train/val s'est-il réduit ? De combien ?
- La convergence est-elle plus lente ou plus rapide ? Pourquoi ?
- Qu'est-ce qui reste insuffisant pour votre dataset ? Pourquoi l'augmentation seule ne suffit pas ?

Trois scenarios pour blinder cette phase :

- Happy path : graphe de superposition sauvegardé, chiffres clés affichés, paragraphe d'interprétation écrit dans une cellule Markdown (pas dans un commentaire Python), TP2 montre `val_acc > TP1` et `gap < TP1`.
- Edge case : comparer sur 5 epochs vs 30 epochs. La conclusion peut changer : l'augmentation aide surtout au-delà d'un certain nombre d'epochs, parce qu'elle ralentit la convergence mais améliore la généralisation à terme.
- Adversarial : relancer TP1 et TP2 sur 3 seeds différentes (`tf.random.set_seed(0)`, 1, 42). L'écart entre seeds est-il plus grand que l'écart TP1 vs TP2 ? Si oui, le gain observé est peut-être dans le bruit statistique, pas dans la régularisation. C'est la question honnête à se poser avant de conclure.

Pensez déjà à votre projet J5. Vous venez de voir trois architectures sur le même problème. En J5, vous choisirez votre propre domaine. Critères : un sujet qui vous tient à coeur, un dataset accessible (Kaggle, HuggingFace), un entraînement faisable en moins de 20 min sur Colab T4, une démo déployable en WebApp. Si votre problème est une image, prenez MobileNetV2 comme point de départ. Si c'est une séquence, ça vient demain. Si c'est du texte structuré, vous avez déjà vu le pipeline en J2. Quelqu'un a déjà une idée de domaine ?

Phase 3.1 : Chargement MobileNetV2 et freezing

Objectif : Charger MobileNetV2 pré-entraîné sur ImageNet, geler la base, construire une tête de classification custom, et vérifier dans `model.summary()` que zéro paramètre de la base n'est entraînable.

Avant de charger : Commit "phase 3.1 MobileNetV2 chargement".

MobileNetV2 attend des images de taille minimum 32x32. La taille recommandée est 160x160 ou 224x224. On reconstruit le pipeline de données avec cette taille pour maximiser la qualité des représentations.

```
# TODO : reconstruire train_ds_tl et val_ds_tl avec IMG_SIZE_TL = (160, 160).
# Même structure que phase 1.2 (image_dataset_from_directory, label_mode='binary', etc.).
IMG_SIZE_TL = (160, 160)
# TODO : créer train_ds_tl avec shuffle=True, seed=42
# TODO : créer val_ds_tl avec shuffle=False, seed=42
```

```
# preprocess_input de MobileNetV2 normalise dans [-1, 1] (pas [0, 1]).
# Ne pas appliquer Rescaling(1./255) ici : preprocess_input le remplace.
preprocess_input = tf.keras.applications.mobilenet_v2.preprocess_input

# TODO : appliquer preprocess_input sur train_ds_tl et val_ds_tl avec .map() +
# .prefetch(AUTOTUNE).
# Même structure que la normalisation en phase 1.2, juste avec preprocess_input à la place
# de Rescaling.
```

```

base_model = tf.keras.applications.MobileNetV2(
    input_shape=(160, 160, 3),
    include_top=False,
    weights='imagenet'
)

# TODO : afficher le nombre de couches dans base_model (len(base_model.layers)).
# TODO : afficher le nombre de paramètres totaux (base_model.count_params()).

# TODO : mettre base_model.trainable = False (geler toute la base)

# TODO : construire le modèle avec l'API fonctionnelle :
# inputs = tf.keras.Input(shape=(160, 160, 3))
# x = base_model(inputs, training=False) # training=False : BatchNorm reste en mode
inférence
# x = tf.keras.layers.GlobalAveragePooling2D()(x)
# x = tf.keras.layers.Dense(128, activation='relu')(x)
# x = tf.keras.layers.Dropout(0.3)(x)
# outputs = tf.keras.layers.Dense(1, activation='sigmoid')(x)
# model_tl = tf.keras.Model(inputs, outputs)

# TODO : model_tl.summary()

```

Devrait afficher : la base MobileNetV2 apparaît comme un bloc unique avec "Non-trainable params" égal à ses paramètres totaux. Seule la tête custom a des paramètres entraînables. Le nombre de paramètres entraînables doit être autour de 165 000.

Verifications :

- Happy path : `model_tl.summary()` montre la base MobileNetV2 frozen (Non-trainable params ~2.2M, Trainable params autour de 165 000 pour la tête custom), output shape finale `(None, 1)`.
- Edge case : charger MobileNetV2 sans `weights='imagenet'`. Les poids sont aléatoires, le transfer learning n'apporte rien. Comparer `val_accuracy` après 5 epochs avec et sans pretrained pour mesurer concrètement l'apport des représentations ImageNet.
- Adversarial : charger MobileNetV2 avec `include_top=True` puis essayer de plugger une tête custom dessus. Erreur de shape car la tête ImageNet sort 1000 classes, pas l'embedding voulu.
`include_top=False` est obligatoire dès qu'on remet une tête custom.

Phase 3.2 : Entraînement de la tête uniquement

Objectif : Entraîner uniquement la tête de classification (base gelée) pendant 10 epochs, observer la convergence rapide, et vérifier que la `val_accuracy` dépasse 90%.

Avant d'optimiser : Commit "phase 3.2 head training".

```
# TODO : compiler model_tl avec Adam(learning_rate=1e-3), loss='binary_crossentropy',
metrics=['accuracy'].
# La base est gelée : on peut se permettre un lr élevé pour la tête.

# TODO : créer log_dir_tl et tensorboard_callback_tl (log dans "logs/transfer/...")

# TODO : lancer model_tl.fit(train_ds_tl, epochs=10, validation_data=val_ds_tl,
#                             callbacks=[tensorboard_callback_tl])
# Stocker dans history_tl_head, mesurer training_time_head.

print(f"Temps d'entraînement (tête seule) : {training_time_head:.0f}s")
print(f"val_accuracy finale (tête seule) :
{max(history_tl_head.history['val_accuracy']):.3f}")
```

```
plot_history(history_tl_head, "Transfer Learning - tête seule")
```

Ce que vous devez observer : la val_accuracy dépasse 90% en 3-4 epochs sur la plupart des datasets visuels. La convergence est rapide car la base connaît déjà les représentations utiles. Il y a presque pas d'overfitting : la base ne bouge pas, seule la tête (peu de paramètres) s'adapte.

Trois cas à valider :

- Happy path : val_accuracy dépasse 90% en 3-5 epochs, convergence visiblement plus rapide que TP1/TP2. Train et val convergent ensemble, gap quasi inexistant.
- Edge case : entraîner avec `learning_rate=1e-1` au lieu de 1e-3. La val_accuracy oscille violemment. La tête bouge trop à chaque batch, le modèle ne converge pas vers un optimum stable.
- Adversarial : oublier `base_model.trainable = False` avant de compiler. Toutes les couches s'entraînent avec lr=1e-3, le pretrained se détériore en 2-3 epochs, la val_accuracy redescend sous les résultats de TP1. C'est le piège classique du transfer learning.

Phase 3.3 : Fine-tuning des dernières couches

Objectif : Dégeler les dernières couches de MobileNetV2, réentraîner avec un learning rate réduit, et gagner 2 à 5 points de val_accuracy supplémentaires.

Avant de fine-tuner : Commit "phase 3.3 fine-tuning".

```
# Les premières couches (bords, textures basiques) sont universelles : pas besoin de les
retoucher.
# Les dernières couches (formes complexes, concepts) sont adaptées au domaine cible.

base_model.trainable = True

print(f"Nombre de couches dans la base : {len(base_model.layers)}")

# TODO : calculer fine_tune_at = 80% du nombre de couches (int).
# Heuristique : geler les 80% premières, dégeler les 20% dernières.
fine_tune_at = # TODO : int(len(base_model.layers) * 0.8)

# La boucle de dégel est donnée (boilerplate Keras standard).
```

```

for layer in base_model.layers[:fine_tune_at]:
    layer.trainable = False

print(f"Couches dégelées : {len(base_model.layers) - fine_tune_at}")
trainable_count = sum(p.numpy().size for p in model_tl.trainable_variables)
print(f"Paramètres entraînaables : {trainable_count:,}")

# TODO : recompiler model_tl avec Adam(learning_rate=1e-4).
# lr x10 plus petit que pour la tête seule : les poids de la base encodent des
représentations
# valides d'ImageNet. Un grand lr les écraserait (catastrophic forgetting).

# TODO : créer early_stopping_tl (monitor='val_accuracy', patience=5,
restore_best_weights=True)

# TODO : lancer model_tl.fit(train_ds_tl, epochs=15, validation_data=val_ds_tl,
#                             callbacks=[tensorboard_callback_tl, early_stopping_tl])
# Stocker dans history_tl_finetime, mesurer training_time_finetime.

print(f"Temps d'entraînement (fine-tuning) : {training_time_finetime:.0f}s")
print(f"val_accuracy finale : {max(history_tl_finetime.history['val_accuracy']):.3f}")

plot_history(history_tl_finetime, "Transfer Learning - fine-tuning")

```

Et là, le gain est visible 🤩 C'est ça, le transfer learning. Le même problème, le même nombre d'images, le même Colab T4. Mais un modèle qui a déjà vu 14 millions d'images avant d'arriver. La différence n'est pas dans votre code : elle est dans les représentations.

Quality gate :

- Happy path : val_accuracy gagne 2-5 points par rapport à la phase 3.2. Pas de divergence, courbes plus stables que TP1.
- Edge case : dégeler 100% des couches du base_model avec lr=1e-5. Théoriquement valide mais le risque de détruire les premières couches est réel. Mesurer la val_acc et comparer avec le dégelage partiel (80% gelées) de la phase principale.
- Adversarial : dégeler avec `learning_rate=1e-3` (pas réduit, le même que pour la tête). Le pretrained se casse en 2 epochs, val_acc chute sous la baseline phase 3.1. C'est exactement pourquoi on baisse le lr pour le fine-tuning.

Bonus perso J3. MobileNet tourne en temps réel sur un téléphone. Ce que vous venez de fine-tuner pourrait tourner dans une app mobile avec [TensorFlow Lite](#). L'export prend 10 lignes de code, le fichier `.tflite` fait moins de 10 Mo. Si ça vous intrigue : c'est exactement ce que fait la phase 3.5. Et si vous avez une alternance en mobile ou IoT, c'est une démo qui se distingue.

Phase 3.4 : Tableau de comparaison cross-modèles

Objectif : Extraire les métriques clés des trois modèles et produire le tableau de comparaison complet qui documente la progression de la journée.

Avant de compiler : Commit "phase 3.4 comparaison cross-modèles".

```
import os

# TODO : sauvegarder les trois modèles.
# model_scratch.save("/content/model_scratch.keras")
# model_augmented.save("/content/model_augmented.keras")
# model_tl.save("/content/model_tl.keras")

def model_size_mb(path):
    return os.path.getsize(path) / (1024 * 1024)

size_scratch = model_size_mb("/content/model_scratch.keras")
size_augmented = model_size_mb("/content/model_augmented.keras")
size_tl = model_size_mb("/content/model_tl.keras")

# TODO : extraire les métriques suivantes pour chaque modèle :
# val_acc_scratch = max(history_scratch.history['val_accuracy'])
# val_acc_augmented = ...
# val_acc_tl = max des deux phases 3.2 et 3.3 combinées
# (max(history_tl_head.history['val_accuracy'] +
history_tl_finetune.history['val_accuracy']))
# params_scratch = model_scratch.count_params()
# ... (idem pour les deux autres)

print("=" * 70)
print("TABLEAU DE COMPARAISON DES TROIS MODELES")
print("=" * 70)
print(f"{'Iteration':<25} {'val_acc':>8} {'Params':>12} {'Temps':>8} {'Taille':>8}")
print("-" * 70)

# TODO : afficher une ligne par modèle avec ces 5 colonnes.
# Format : print(f"{'CNN scratch':<25} {val_acc_scratch:>7.1%} {params_scratch:>12,}
# {time_s:>7.0f}s {size_mb:>7.1f}Mo")

print("=" * 70)
```

Devrait afficher quelque chose comme :

```
=====
```

Iteration	val_acc	Params	Temps	Taille
CNN scratch	68.4%	6,243,553	710s	72.1Mo
CNN augmenté + Dropout	83.7%	6,243,553	860s	72.1Mo
MobileNetV2 fine-tuning	94.2%	4,387,521	330s	16.2Mo

```
=====
```

Les chiffres varient selon votre dataset. Ce qui compte : la progression d'une colonne à l'autre, et le fait que MobileNetV2 est plus léger ET plus performant. 🌟

```
# TODO : tracer la superposition des val_accuracy des trois modèles sur un seul graphe.
# Structure :
# plt.plot(history_scratch.history['val_accuracy'], label='CNN scratch', color='red',
# linestyle='--')
# plt.plot(history_augmented.history['val_accuracy'], label='Augmenté + Dropout',
# color='orange')
# tl_acc = history_tl_head.history['val_accuracy'] +
# history_tl_finetune.history['val_accuracy']
# plt.plot(range(len(tl_acc)), tl_acc, label='MobileNetV2 fine-tuning', color='green')
# plt.xlabel('Epoch') ; plt.ylabel('Val Accuracy') ; plt.legend()
# plt.savefig("comparison_all.png", dpi=100)
```

Dans une cellule Markdown, écrivez un paragraphe d'interprétation : qu'est-ce que le tableau révèle sur le rapport performance/coût de chaque approche ? Pourquoi MobileNetV2 gagne sur tous les fronts malgré moins de paramètres actifs ?

Checkpoint transfer learning :

- Happy path : tableau à 3 lignes complet avec 5 colonnes, graphe de superposition sauvegardé, paragraphe d'interprétation en cellule Markdown. Progression visible entre les trois modèles.
- Edge case : ajouter une colonne "temps par epoch" plutôt que "temps total". MobileNetV2 a moins de temps par epoch que le CNN scratch malgré 2.2M de paramètres frozen, parce que la backprop ne traverse pas la base. Ce détail change la lecture du tableau.
- Adversarial : comparer val_acc sans documenter les paramètres entraînaables ni le temps. Un tableau "95% gagne" sans contexte fait croire que TP3 est magique, mais TP3 utilise 2.2M params pretrained vs TP1 6M params from scratch. Sans la colonne params, le tableau ment par omission.

Phase 3.5 : Export TFLite et exploration ouverte

Objectif : Convertir le modèle fine-tuné en format TFLite, mesurer la compression, puis explorer librement (quantization, architecture alternative, push de précision, autre dataset).

Avant d'exporter : Commit "phase 3.5 export tflite".

Il n'y a pas d'état "terminé" ici. Cette phase n'a pas de plafond.

Export TFLite

[TensorFlow Lite](#) est le format de déploiement de TensorFlow pour les appareils embarqués et mobiles. La conversion prend le modèle `.keras` et produit un fichier `.tflite` optimisé.

```
# TODO : créer le convertor.
# convertor = tf.lite.TFLiteConverter.from_keras_model(model_t1)
convertor = # TODO

tflite_model = convertor.convert()

# Adapter le nom du fichier à votre dataset (pas "cats_dogs")
tflite_path = f"/content/{CLASS_A}_vs_{CLASS_B}_mobilenet.tflite"
with open(tflite_path, "wb") as f:
    f.write(tflite_model)

# TODO : afficher la taille du .tflite et du .keras, calculer le facteur de compression.
# size_tflite = os.path.getsize(tflite_path) / (1024 * 1024)
# size_keras = model_size_mb("/content/model_t1.keras")
# print(f"Taille Keras : {size_keras:.1f} Mo")
# print(f"Taille TFLite : {size_tflite:.1f} Mo")
# print(f"Facteur de compression : {size_keras / size_tflite:.1f}x")
```

Inférence avec le modèle TFLite

```
# TODO : charger l'interpréteur TFLite et lancer une inférence sur une image du val set.
# Structure :
# interpreter = tf.lite.Interpreter(model_path=tflite_path)
# interpreter.allocate_tensors()
# input_details = interpreter.get_input_details()
# output_details = interpreter.get_output_details()
# # récupérer une image depuis val_ds_t1 : images[0:1].numpy()
# interpreter.set_tensor(input_details[0]['index'], test_image)
# interpreter.invoke()
# prediction = interpreter.get_tensor(output_details[0]['index'])[0][0]
# print(f"Prédiction : {prediction:.3f} (0={CLASS_A}, 1={CLASS_B})")
# print(f"Vraie classe : {CLASS_A if true_label == 0 else CLASS_B}")
# print(f"Confiance : {max(prediction, 1-prediction):.1%}")
```

Directions d'exploration (choisissez ce qui vous intéresse)

Direction A : quantization INT8. Plus agressive que la conversion FP32 par défaut, elle convertit les poids en entiers 8 bits. Taille réduite de 4x environ, avec une légère perte de précision.

```
converter_int8 = tf.lite.TFLiteConverter.from_keras_model(model_t1)
converter_int8.optimizations = [tf.lite.Optimize.DEFAULT]

# TODO : fournir un dataset de calibration pour estimer les plages de valeurs des
# activations.
# representative_dataset est une fonction générateur qui yield des batches numpy.
# Exemple de structure :
```

```
# def representative_dataset():
#     for images, _ in val_ds_tl.take(50):
#         yield [images.numpy()]
# Puis : converter_int8.representative_dataset = representative_dataset

tflite_int8 = converter_int8.convert()

tflite_int8_path = f"/content/{CLASS_A}_vs_{CLASS_B}_mobilenet_int8.tflite"
with open(tflite_int8_path, "wb") as f:
    f.write(tflite_int8)

# TODO : comparer size_int8 vs size_tflite (FP32). Mesurer l'accuracy_drop avec
# l'interpréteur quantisé.
```

Direction B : architecture alternative. Remplacez MobileNetV2 par EfficientNetB0, VGG16 ou ResNet50. Répétez les phases 3.1 à 3.3. Ajoutez une ligne dans le tableau de comparaison. Est-ce que le gain en précision vaut le coût en taille et temps ?

```
# TODO : charger une architecture alternative avec include_top=False, weights='imagenet'.
# Adapter l'input_shape si nécessaire (VGG16 et ResNet50 recommandent 224x224).
# Construire la même tête de classification (GlobalAveragePooling2D + Dense(128) + Dropout
+ Dense(1)).
# Comparer val_accuracy, temps d'entraînement, taille du modèle sauvegardé.
```

Direction C : push de la précision. Essayez de maximiser la val_accuracy sur votre dataset avec MobileNetV2. Pistes : dégeler plus de couches, réduire encore le learning rate, augmenter l'input à 224x224, ajouter `ReduceLROnPlateau`, ajuster le seuil de Dropout.

Direction D : autre dataset. Testez le même pipeline sur un second sujet de classification binaire. Choisir un domaine visuellement différent de votre premier dataset. Qu'est-ce qui change dans les hyperparamètres optimaux ? Qu'est-ce qui reste identique ?

Verifications avant export :

- Happy path : fichier `.tflite` produit avec le bon nom, taille affichée, facteur de compression calculé, une inférence TFLite exécutée avec succès et résultat affiché, au moins une direction d'exploration entamée et commitée.
 - Edge case : Direction A, quantization INT8. La taille du modèle est divisée par ~4 mais la val_accuracy peut perdre 1-2 points sur le val set. Mesurer l'accuracy_drop avec l'interpréteur TFLite quantisé pour décider si le tradeoff vaut le coup sur votre dataset.
 - Adversarial : tester le modèle sur une image hors distribution (un sujet qui n'appartient ni à CLASS_A ni à CLASS_B). Le modèle binaire force une prédiction avec une proba souvent élevée. Aucun mécanisme "rien de connu" : c'est une limite architecturale à documenter dans le README, pas un bug.
-

Recapitulatif Jour 3

Ce qu'on a appris

1. **Convolutions** : localité (chaque neurone voit un patch), partage de poids (un kernel scanne toute l'image), invariance de translation (le chat à gauche et à droite activent les mêmes filtres). La solution naturelle aux limitations du PMC sur les images.
2. **Architecture CNN** : calculer les shapes à la main avec la formule $\text{floor}((W - K + 2P) / S) + 1$, comprendre padding, stride, pooling, Flatten. Plus de boîte noire Keras.
3. **Data augmentation + Dropout** : créer de la diversité synthétique pour régulariser, intégration comme couches Keras modernes (actives en training, passives en inférence), placement du Dropout après Flatten.
4. **Transfer learning** : poids pré-entraînés sur ImageNet, stratégies freezing total / fine-tuning partiel / fine-tuning complet. MobileNetV2 comme défaut pour le mobile et l'embarqué.
5. **Le contraste des trois itérations** : CNN scratch ~70%, augmentation + dropout ~85%, MobileNetV2 fine-tune ~95%. Le modèle le plus léger est le plus performant.
6. **TFLite** : un modèle entraîné peut être compressé et déployé sur mobile ou IoT en quelques lignes de code.
7. **Keras vs PyTorch** : même logique, deux dialectes. Keras masque la boucle d'entraînement, PyTorch l'expose.

Points d'attention pour la prochaine session

- En J4, on quitte les images statiques pour les séquences temporelles. Un réseau qui "lit" une série de valeurs dans l'ordre plutôt qu'un tableau de pixels. La structure est différente, mais l'API Keras reste la même.
- Le déploiement WebApp arrive en J4 : pensez à ce que serait une WebApp qui prend une image en entrée et renvoie "chat" ou "chien". On le fera avec du texte et des séquences, mais le principe est identique.
- **Projet J5** : si votre projet implique de la vision, vous avez maintenant tout le scaffold nécessaire. Affinez votre idée d'ici là.